

Document de fin d'itération 2

Contents

1 Problèmes résolus et choix de conceptions	1
1.1 Développement hors connexion	1
1.2 Line-ending (CRLF/LF)	1
1.3 Repositories	1
1.3.1 Tests	2
1.4 Refactoring : vue & controller	2
1.5 Documentation générée par IA	2
1.6 Refactoring CSS	2
1.7 Intérêt des DTOs ?	2
2 Problèmes pas encore résolus	2
2.1 Code des animations qui apparaît dans AutomaticCombatController	2
2.2 Préparation de contrôleurs	2

1 Problèmes résolus et choix de conceptions

1.1 Développement hors connexion

Problème : instance Postgres sur un serveur distant → impossible de travailler sans connexion internet.

Solution : instance postgres dans un container docker.

1.2 Line-ending (CRLF/LF)

Problème : line ending différents parce que certains d'entre nous sont sur Windows et d'autres sont sur des systèmes Unix-like et d'autres sur Windows.

Windows	Unix-like
CRLF	LF

Table 1: line-ending Windows et Unix-like

Conséquence : des git diff qui affichent que des fichiers entiers ont été modifiés même quand seul quelques lignes ont été modifiées en réalité.

Solution :

- outil dos2unix pour convertir toute la codebase en LF.
- modification du .clang-format pour forcer LF (pour les prochains commits).

1.3 Repositories

Pour réaliser les histoires de sauvegarde et modification des équipes, nous avons utilisé une base de données postgres. Pour cela, nous avons utilisé le repository pattern.

- DatabaseRepository : pour déléguer à UserRepository et StaticDataRepository
- UserRepository : données modifiables liées aux joueurs (bugemon et équipes des utilisateurs notamment)
- StaticDataRepository : pour la gestion des données persistantes

Note : après réflexion, nous trouvons cette architecture bizarre. Nous pensons qu'il serait plus logique d'injecter un objet servant à charger les queries dans UserRepository et StaticDataRepository.

1.3.1 Tests

Utilisation de la bibliothèque mockito pour tester les repositories.

1.4 Refactoring : vue & controller

Après avoir échangé avec Pr. Frédéric Pluquet avant le cours du 17 mars à propos de notre architecture MVC, nous avons constaté que celle-ci pouvait être améliorée.

Améliorations apportées :

- utilisation de callbacks : Runnable, Consumer, Function
- utilisation d'interfaces pour cacher l'implémentation de la vue (au controller)

1.5 Documentation générée par IA

Durant l'itération 1, nous avons généré la majorité de notre javadoc avec une IA. Au départ, le résultat nous a paru très satisfaisant.

A la fin de cette itération 2, nous avons changé de point de vue : nous pensons que cette documentation est beaucoup trop exhaustive pour un projet de cette envergure. Nous n'avons pas assez d'heures par semaine pour pouvoir suffisamment avancer sur le code tout en maintenant celle-ci à jour.

De plus, cette javadoc mentionne tellement de classes que lorsqu'on effectue une requête LSP pour chercher les références, on perd du temps à distinguer les références du symbole qui sont réellement du code et celles de la javadoc.

1.6 Refactoring CSS

Nous avons refactorisé (à coup d'IA) le CSS pour éviter la répétition (notamment celles des couleurs) avec un fichier `theme.css`.

1.7 Intérêt des DTOs ?

Nous utilisons actuellement des DTOs, ceux-ci sont implémentés comme des interfaces pour les objets du modèle.

Ceux-ci nous semblent inutiles : nous avons l'impression que "poser des questions au modèle" directement depuis la vue est plus simple, tout en n'introduisant aucun inconvénient par rapport à notre utilisation actuelle des DTOs (interfaces).

2 Problèmes pas encore résolus

2.1 Code des animations qui apparaît dans AutomaticCombatController

AutomaticCombatController contient un `javafx.animation.Timeline`. Nous réfléchissons à une solution pour ne pas garder cet objet `javafx` dans un controller.

2.2 Préparation de contrôleurs

Nous pensons que nous avons besoin d'un système permettant de "préparer" les contrôleurs pour les transitions.

Exemple : lorsque le CombatController réalise qu'un combat est terminé,

1. celui-ci doit switch vers le LevelUpController (et sa vue). Mais il lui faut un moyen de fournir la liste des LevelUp qui doivent être affichés (pour laisser l'utilisateur choisir les upgrades sur les Bugeons).

2. Une fois que tous les LevelUp ont été affichés, le LevelUpController devrait pouvoir rappeler un callback de fin, qui serait créé par le CombatController et switcherait vers la vue de victoire ou défaite. Nous n'avons actuellement pas de moyen de passer un tel callback proprement.